Homework 1

20252028 – Assylzhan Tunggushbay
Documentation

2 – Introduction

3 – How it works

3 – Memory Management and File Parcing

4 – Binary To Hex

5 – Endian Conversion

5 – Data Type Conversion

9 – Convenience

10 - Example

Introduction

This project implements low-level bit operations and imitates main memory to represent different data types. Specifically, this project reads input of bits from text file, stores it in 3D Tensor in the form 64-bit chunks, converts them to various data types and outputs to text file.

Objectives:

- Read binary data from text file , Write to text file
- Convert binary to hex to other data types
- Show Endianness handling , bitwise operations , memory management

As core concepts project uses:

- bit shifting (<< , |)
- Endian converting
- Dynamic memory allocation (malloc , realloc , free)
- Reading from and writing to text files (fopen , fgetc , fprintf , fclose)
- Data representation (ASCII , signed/unsigned characters , signed/unsigned integers , signed float , signed double)

Process Flow:

1) Input.txt is read
2) Bits are stored in 3D Tensor
3) Binary to Hex conversion
4) Endian Adjustment
5) Type conversion
6) Writing in output.txt

Structure:

- hw1.c - main program
- hw1.h – header linking all modules
- arrop.c – Dynamic array management (stands for array operations)
- converter.c – All data type conversions
- output.c – Output manager
- input.txt – Input binary data
- output.txt – Generated output
- Makefile – convenient way to work with compilation

How it works

1. Memory Management and File Parsing

```
int layer = 1; //initial size of memory

// Creation of 3D array (Tensor) with dynamic layer size , row size of 8 and column size of 8.
// Basically , dividing memory into 64 bit chunks.
char ***chunks_of_64bits;
chunks_of_64bits = CreateArr(chunks_of_64bits, layer , 8 , 8);
```

3D array is created where dynamic layer is outside, meaning chunks of 64-bits are dynamic, rest are not. Inner layer contains of 8 arrays of 8 bits , summing in 64 bits. Tensor is created with CreateArr() function that uses initialized array and its sizes as arguments , and it returns new modified array.

```
//creates dyanamic 3D array
char ***CreateArr(char ***array , int size1 , int size2 , int size3){
    //increases size of array in runtime , using malloc
    array = (char ***)malloc(size1 * sizeof(char **));
    // triple pointer means layer , double - row , single - column and without - the value
    for (int i = 0; i < size1; i++) {
        array[i] = (char **)malloc(size2 * sizeof(char *));
        for (int j = 0; j < size2; j++) {
            array[i][j] = (char *)malloc(size3 * sizeof(char));
        }
    }

    return array;
}
```

Specifically, it uses malloc() to each layer of array. Initial 64-bit layer size is equal to 1. After FiletoArr() function is called , array is dynamically filled with input bits in a runtime. Each 64 bit is read, layer size is increased to meet memory requirements.

```
char ***FileToArr(char ***array , FILE *file_pointer , int *layer){

    int buffer_size = 64;//current size of memory
    int index = 0;//current number of elements in memory
    int ch;//placeholder of read data from file
    int i = 0; int j = 0; int k = 0; //indexes of 3D array

    //loop that places data from file into array
    while((ch = fgetc(file_pointer)) != EOF){ //reads character one by one until file is empty

        if (index >= buffer_size){
            (*layer)++; //increasing layer size of 3D array , adding additional chunk of 64 bit
            buffer_size += 64;
```

FileToArr() uses fgetc() to read each character (including newline, which is skipped) and fill array, and it stops only when End Of File is encountered.

```
//updating size of array, using realloc
char ***temp;
temp = realloc(array , *layer * sizeof(char **));
array= temp;


array[*layer-1] = malloc(8 * sizeof(char *));
for (char x = 0; x < 8; x++){
    array[*layer-1][x] = malloc(8 * sizeof(char));
}
```

After memory requirements are checked , if 64 bits are already encountered , array size is updated , using realloc(), and the malloc() to each new layer. Process of filling the array is continued.

After everything is done Dynamic Memory should be freed and files must be closed.

```
//freeing memory in the heap and closing files
FreeArr(chunks_of_64bits, layer , 8);
FreeArr(chunks_of_16hex , layer , 8);
fclose(output);
fclose(file_pointer);
```

2. Binary To Hex

As chunks_of_64bits is filled with bits , they are converted to hex values.

```
//Creation of Tensor , where previous Binary Tensor will be saved as hex values
char ***chunks_of_16hex;
chunks_of_16hex = CreateArr(chunks_of_16hex , layer , 8 , 2);//4 bits = 1 hex , so 8 bits = 2 hex
```

New 3D Tensor for hex values is created with CreateArr() , where outer layer size is already known. However , now inner layer consists of 8 arrays of 2 hexes , as 8 bits are equal to 2 hexes.

```
int value = 0;
for(int i = 0; i < layer; i++){
    for(int j=0; j<8; j ++){
        for(int k=0; k<8; k+=4){ //skipping 4 indexes , as 4 bits = 1 hex
            value = 0;//holder of conversion form binary to decimal
            for(int a = 0; a<4; a++){ //4th loop to compensate skipping.

                //bit shifting to convert from array of bits to decimal
                value = (value << 1) | (chunks_of_64bits[i][j][k+a] - '\0') - 48;
            }
            chunks_of_16hex[i][j][k/4] = DecimalToHex(value);

        }

    }
}
```

Hex Tensor is filled by tracing through Binary Tensor. Binaries are converted to Hex by bitwise operation shown in the figure above. First, bits are converted from string value to integer value; after which, they are shifted into value holder. As 4 bits are shifted into value holder, value is used in DecimalToHex() function that returns string of equivalent hex value (more detailed about function later), and then stored in corresponding index in Hex Tensor.

3. Endian Conversion

As a requirement of task , endian need to be changed from Big Endian to Small Endian.

```
void ChangeEndian(char ***array , int layer) {
    for(int j = 0; j < layer; j++){

        for (int i = 0; i < 4; i++) {
            char *temp = array[j][i];
            array[j][i] = array[j][7 - i];
            array[j][7 - i] = temp;
        }
    }

}
```

ChangeEndian() is simple function that swaps places of opposing inner layers of 2 hexes. 64-bit chunks (16-hex chunks in our cases) and places of hexes themselves are not changed. Only endian is changed.

4. Data Type Conversion
   - Hex To Decimal and Decimal To Hex

```c
//maps index value to hex. For example 10th index is A
char DecimalToHex(int value){
    //given value is considered as index
    char HexMap[] = "0123456789abcdef";
    return HexMap[value];
}

//does opposite operation. Uses property of char being integer value.
int HexToDecimal(char c) {
    if (c >= '0' && c <= '9') return c - '0';
    if (c >= 'a' && c <= 'f') return c - 'a' + 10;
    if (c >= 'A' && c <= 'F') return c - 'A' + 10;
    return 0;
}
```

DecimalToHex() function uses value mapping , meaning it uses value as index to corresponding hex value. For example , value = 12 , so index = 12 and HexMap[12] = c .

HexToDecimal() function uses property of characters being integer numbers. By adding and subtracting numbers you can change character , and by adding or subtracting characters you can change number.

- Decimal to ASCII

```c
//Prints ASCII values to the output file
void DecimalToASCII(char ***array , int size1 , int size2 , FILE *output){
    fprintf(output, "[*] ASCII Codes: ");
    for (int i = 0; i < size1; i++) {
        for (int j = 0; j < size2; j++) {

            int high = HexToDecimal(array[i][j][0]); //first 4 bits
            int low  = HexToDecimal(array[i][j][1]); //latter 4 bits

            int value = (high << 4) | low; //bit shift operation to store 8 bits (1 byte)

            //checks if byte has ASCII representation
            if(value >= 32 && value <= 126){
                fprintf(output , "%c ", value);
            }
            else{
                fprintf(output , ". ");
            }


        }
    }
    fprintf(output , "\n");
```

DecimalToASCII() function uses 8 inner layers of 2 hexes to print out 8 ASCII values corresponding to those hex values. Conversion is done by bit shifting, explained

earlier. Not all bits have printable ASCII values , that is why function checks the range of the bit shifted value. If it has corresponding ASCII character it prints it on the output text via “%c” , and if it has not it prints “.” symbol.

From this function every conversion function prints its data type on output.txt via output = fopen(“input.txt” , “w”); and fprintf(output , “text” , variable);

- Decimal To Unsigned and Signed characters

```
void DecimalToSC(char ***array ,int size1 ,int size2 , FILE *output){
    fprintf(output , "[*] Signed Char: ");
    for (int i = 0; i < size1; i++) {
        for (int j = 0; j < size2; j++) {

            int high = HexToDecimal(array[i][j][0]);
            int low  = HexToDecimal(array[i][j][1]);

            int value = (high << 4) | low;

            fprintf(output , "%d " , value);

        }
    }
    fprintf(output , "\n");
}
```

```
//Prints Unsigned Character values to Output Files. Similar method to ASCII and Signed Character printing
void DecimalToUC(char ***array ,int size1 ,int size2 , FILE *output){
    fprintf(output , "[*] Unsigned Char: ");
    for (int i = 0; i < size1; i++) {
        for (int j = 0; j < size2; j++) {

            int high = HexToDecimal(array[i][j][0]);
            int low  = HexToDecimal(array[i][j][1]);

            int value = (high << 4) | low;

            fprintf(output , "%u " , value);

        }
    }
    fprintf(output , "\n");
}
```

DecimalToSC() and DecimalToUC() use same principle as ASCII character conversion, but ,as output parameter, they use “%d” and “%u” , respectively.

- Decimal to Signed and Unsigned Integer

```c
//Prints Signed Integer values to Output Files.
void DecimalToS_INT(char ***array , int layer , FILE *output){
    fprintf(output , "[*] Signed Int: ");
    //traces through 64-bit chunks
    for (int i = 0; i < layer; i++){
        //groupping 64 bits into 32-bit chunks
        for(int j = 3; j < 8; j+=4){

            signed int value = 0;//storage of bits
            //considers every 8-bit in 32-bit chunk.8 bits are stored 4 times , resulting in 32 bit value.
            for(int k = 0; k < 4; k++){
                //each hex is given to char, as holders of 4 bits
                char high = HexToDecimal(array[i][j-k][0]);
                char low = HexToDecimal(array[i][j-k][1]);
                value = (value << 4) | high; value = (value << 4) | low;//bit shift operation
            }
            fprintf(output , "%d " , value);
        }
    }
    fprintf(output , "\n");
```

DecimalToS_INT() function uses same bit shifting principle as before , but groups bits to 32-bit groups before implementing shifting. Groupping is done bottom to up , as we use Small Endian. That is why, second iteration starts from 3 , but third iteration is used to decrement its value for index. Value holder also has size of 32-bits minimum. DecimalToU_INT() function does the same thing for unsigned integer , by using parameter "%u"

- Decimal to Float

```c
void DecimalToS_FLOAT(char ***array , int layer , FILE *output){
    fprintf(output , "[*] Signed Float: ");
    //traces through 64-bit chunks
    for (int i = 0; i < layer; i++){
        //groups into 32-bit chunks by skipping 4 indexes
        for(int j = 3; j < 8; j+=4){

            signed int value = 0;//storage for integer bits
            float fvalue; //storage for float bits

            //considers every 8-bit in 32-bit chunk.8 bits are stored 4 times , resulting in 32 bit value.
            for(int k = 0; k < 4; k++){
                char high = HexToDecimal(array[i][j-k][0]);
                char low = HexToDecimal(array[i][j-k][1]);
                value = (value << 4) | high; value = (value << 4) | low;





            }
            fvalue = *(float *)&value; //copies bits of integer to float
            fprintf(output , "%.4f " , fvalue);


        }

    }
    fprintf(output , "\n");
}
```

DecimalToS_FLOAT() function does the same grouping , same bit shifting as DecimalToS_INT() and DecimalToU_INT() , but changes type of value holder to read bits as float, not as integer.

- Decimal to Double

```c
//Prints Signed Double values to Output Files. Similar to printing Signed Float , but uses long long int.
void DecimalToS_DOUBLE(char ***array , int layer , FILE *output){
    fprintf(output , "[*] Signed Double: ");
    for (int i = 0; i < layer; i++){
        long long value = 0; //sizeof(long long int); #8 (64 bits)
        double df;
        //using all 64 bits. Going from high index to small ,as it is small endian.
        for (int j = 7; j >= 0 ; j--){
            char high = HexToDecimal(array[i][j][0]);
            char low = HexToDecimal(array[i][j][1]);
            value = (value << 4) | high; value = (value << 4) | low;//shifts 8 bits 8 times , resulting in storage
        }
        df = *(double *)&value; //copies bits of long long int to signed double
        fprintf(output , "%.4lf " , df);
    }
    fprintf(output , "\n");
}
```

DecimalToS_DOUBLE() function does the same thing as DecimalToS_FLOAT() except it traces through all inner layer from bottom to up , grouping 64-bit chunk into ...well 64-bit chunk. That is because data type double uses 64-bit.

5. Convenience

```c
//Stored for convenience
void Output(char ***array , int layer , FILE *output){
    DecimalToSC(array , layer , 8 , output);
    DecimalToASCII(array , layer , 8 , output);
    DecimalToUC(array , layer , 8 , output);
    DecimalToS_INT(array , layer , output);
    DecimalToU_INT(array , layer , output);
    DecimalToS_FLOAT(array , layer , output);
    DecimalToS_DOUBLE(array , layer , output);
}
```

Output() function grouped every conversion function into one.

```c
#ifndef HW1_H
#define HW1_H

#include "arrop.c"
#include "converter.c"
#include "output.c"

#endif
```

hw1.h serves as header to link every function ,that was created during this project, to main file.

```
hw1:
    gcc hw1.c -o hw1
hw1: hw1.c arrop.c converter.c
    gcc hw1.c -o hw1
clear:
    rm hw1
```

Makefile is easy way to compile the project.

6. Example

```
input.txt
0011111101001010010111100010001001000101010000000011001001001100
0011111101001010010111100010001001000101010000000011001001001100
0011111101001010010111100010001001000101010000000011001001001100
```

Input.txt

```
output.txt
[*] Signed Char: 76 50 64 69 34 94 74 63 76 50 64 69 34 94 74 63 76 50 64 69 34 94 74 63
[*] ASCII Codes: L 2 @ E " ^ J ? L 2 @ E " ^ J ? L 2 @ E " ^ J ?
[*] Unsigned Char: 76 50 64 69 34 94 74 63 76 50 64 69 34 94 74 63 76 50 64 69 34 94 74 63
[*] Signed Int: 1161835084 1061838370 1161835084 1061838370 1161835084 1061838370
[*] Unsigned Int: 1161835084 1061838370 1161835084 1061838370 1161835084 1061838370
[*] Signed Float: 3075.1436 0.7905 3075.1436 0.7905 3075.1436 0.7905
[*] Signed Double: 0.0008 0.0008 0.0008
```

Corresponding output.txt